Filter design:

- *Filtering* is one of the most common forms of signal processing.
- A filter can be equivalently described by $h[n]$, $H(z)$, $H(e^{j\omega}) = |H(e^{j\omega})|e^{j\angle H(e^{j\omega})}$ in magnitude & phase, or $H(e^{j\omega}) = A(\omega)e^{j\phi(\omega)}$ for "signed-magnitude" $A(\omega) \in \mathbb{R}$ and "continuous-phase" $\phi(\omega) \in \mathbb{R}$.
- When designing a filter $H(e^{j\omega})$, we are concerned with
  1. filtering performance:
     - want magnitude response $|H(e^{j\omega})|$ close to target,
     - want phase response $\angle H(e^{j\omega})$ to be "agreeable."
  2. implementational issues:
     - complexity,
     - memory,
     - robustness to quantization effects.
- There are two main classes of digital filter
  1. *recursive* or "*IIR*" (infinite impulse response), and
  2. *non-recursive* or "*FIR*" (finite impulse response).

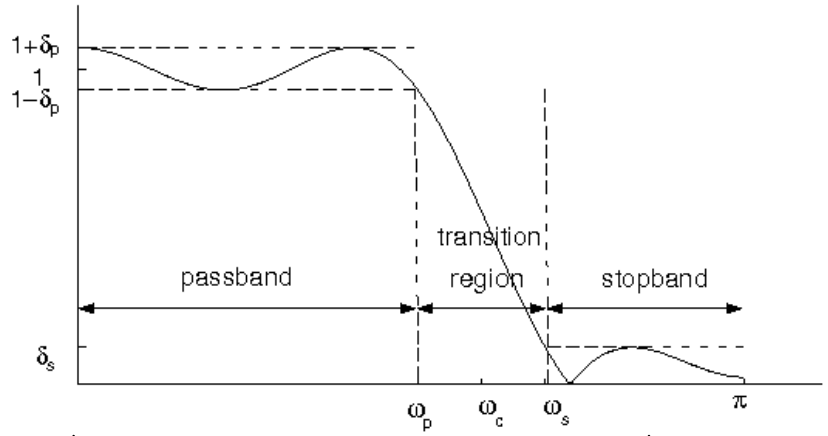
  We will see advantages and disadvantages to each.

Desired magnitude response:

- The desired magnitude response $|D(e^{j\omega})|$ is often a LPF, HPF, BPF, or notch filter,

  though sometimes it is more complicated, e.g., a differentiator, raised-cosine, or Hilbert transformer.

- The error between $|D(e^{j\omega})|$ and the actual response $|H(e^{j\omega})|$ can be measured in different ways:
  - *peak deviations* $\{\delta_p, \delta_s\}$ in passband and stopband:



    or $A_p \triangleq -20\log_{10}(1-\delta_p),\ A_s \triangleq -20\log_{10}\delta_s$ in dB.
  - *squared error*, integrated over ω and possibly weighted:

$$\mathcal{E} = \int_{-\pi}^{\pi} W(\omega)\big(|H(e^{j\omega})| - |D(e^{j\omega})|\big)^2 \,\mathrm{d}\omega$$

Beware: different error criteria lead to different "optimal" filter designs.

Desired phase response:

- Sometimes, we want that $H(e^{j\omega})$ matches a desired response $D(e^{j\omega})$ in phase (as well as magnitude).
- Other times, we want $H(e^{j\omega})$ giving "minimum" delay.
- More often, we desire a "*linear phase*" response

  $$\phi(\omega) = -\omega\ell \quad \text{for some } \ell \in \mathbb{Z},$$

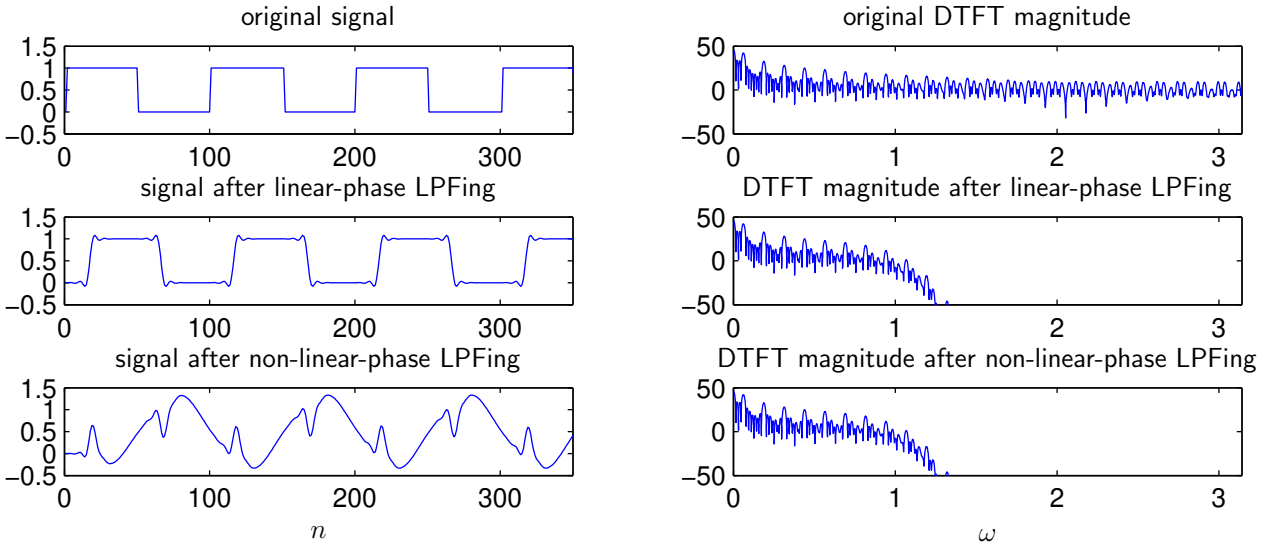
  because the resulting filter

  $$H(e^{j\omega}) = A(\omega)e^{j\phi(\omega)} = A(\omega)e^{-j\omega\ell}$$

  alters the signed-magnitude response by $A(\omega)$ and delays the resulting signal by ℓ samples.

  Trivial phase modification $\Rightarrow$ no phase distortion!

- Phase distortion can have a huge effect on the time-domain characteristics of the filtered signal:



  and such distortion is unwanted in many applications.

Non-recursive filters:

- Non-recursive digital filters typically have the MA form:

$$H(z) = \sum_{k=0}^{N} b_k \, z^{-k} \quad \text{where} \quad \begin{cases} N = \text{“order”} \\ N+1 = \text{“length”} \end{cases}$$

  which results in the difference equation

$$y[n] = \sum_{k=0}^{N} b_k x[n-k]$$

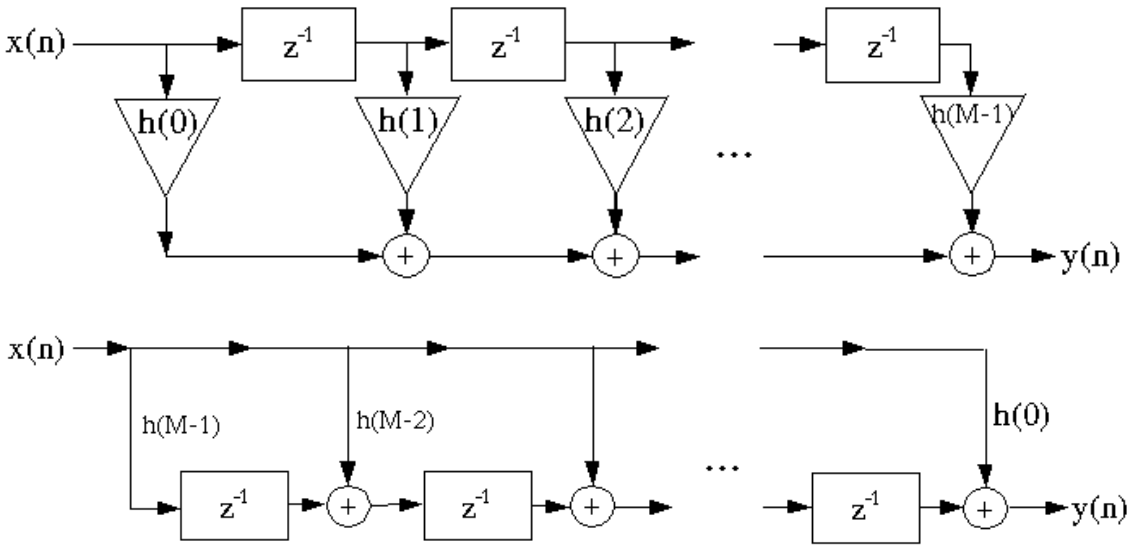
  and thus the (causal) impulse response

$$h[k] = \begin{cases} b_k & \text{for } k = 0 \ldots N \\ 0 & \text{else} \end{cases}$$

- Both memory and complexity grow with N. Since these must be finite, the impulse response length must be too:

  $\Rightarrow$ *“finite impulse response” (FIR) filter*

- Implementation options:



Recursive filters:

- Recursive digital filters typically have the ARMA form:

$$H(z) = \frac{\sum_{k=0}^{N} b_k z^{-k}}{1 + \sum_{m=1}^{M} a_m z^{-m}} \quad \text{with "order"} = \max(M, N)$$

which results in the difference equation

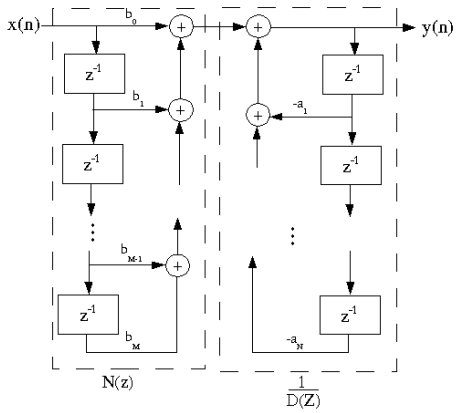
$$y[n] = \sum_{k=0}^{N} b_k x[n-k] - \sum_{m=1}^{M} a_m y[n-m].$$

- The impulse response $h[n]$ has a complicated expression and infinite length.

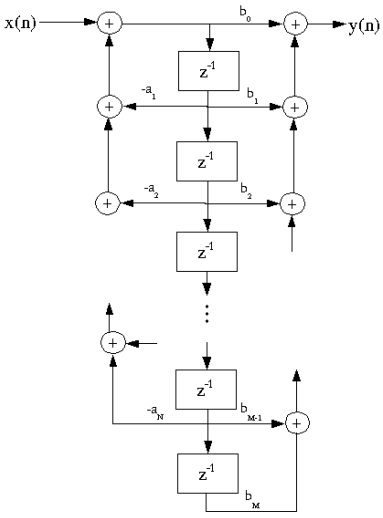
$$\Rightarrow \textit{"infinite impulse response" (IIR) filter}$$

- There are several ways to implement this. While identical in theory, some are more sensitive to quantization.

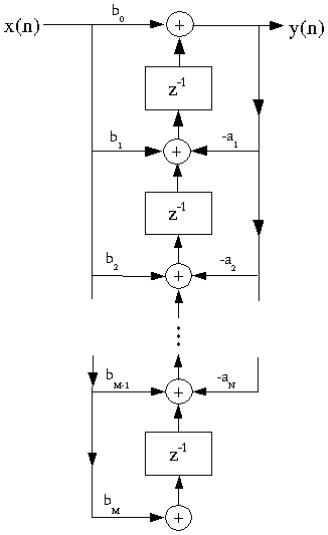
Direct Form I



Direct Form II



Direct Form II transpose



Note that the minimum number of delays is $\max(M, N)$, i.e., the filter order.

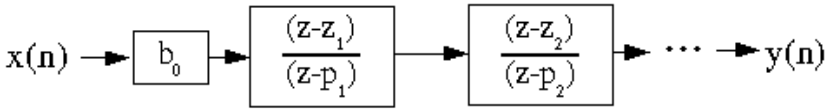
Recursive filters (cont.):

- Another way to write ARMA $H(z)$ is to factor the numerator and denominator as

$$H(z) = \frac{b_0 \prod_{k=1}^{N}(1 - z_k z^{-1})}{\prod_{m=1}^{M}(1 - p_m z^{-1})}$$

with "zeros" $\{z_k\}_{k=1}^{N}$ and "poles" $\{p_m\}_{m=1}^{M}$.

- This factorization suggests the cascade implementation

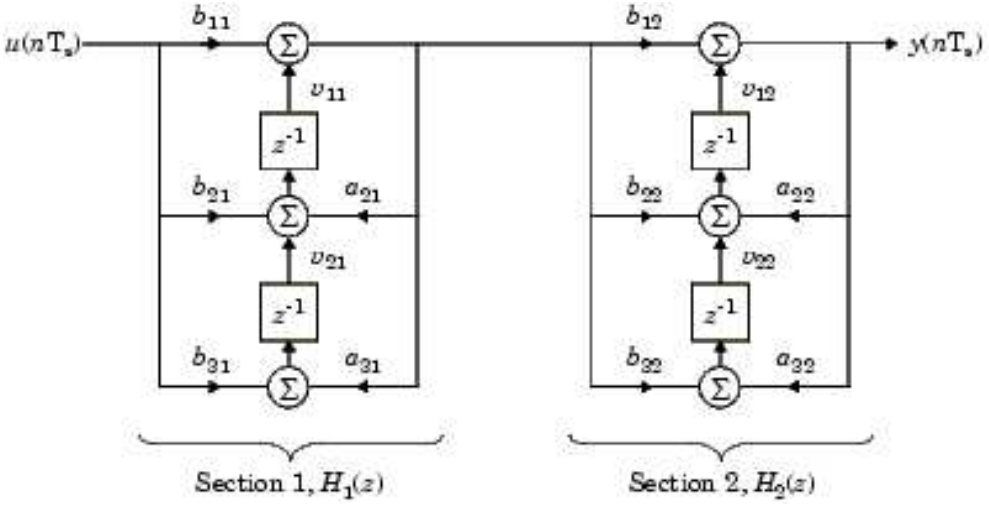


which is very robust to quantization effects.

- When $h[n]$ is real-valued, we can leverage the fact that both $\{z_k\}_{k=1}^{N}$ and $\{p_m\}_{m=1}^{M}$ come in complex conjugate pairs, and cascade real-coefficient 2nd-order sections

$$b_0 \frac{(z - z_n)(z - z_n^*)}{(z - p_n)(z - p_n^*)} = b_0 \frac{z^2 - 2\,\mathrm{Re}\{z_n\} z + |z_n|^2}{z^2 - 2\,\mathrm{Re}\{p_n\} z + |p_n|^2}$$

as follows:



Advantages of FIR and IIR filters:

The advantages of *IIR filters* are:

- much less computation/memory needed to meet target magnitude response,
- better for approximating analog filters,

and the advantages of *FIR filters* are:

- conceptually simpler,
- can have linear phase,
- always stable,
- relatively insensitive to quantization noise,
- easy to make "adaptive,"
- can implement using fast convolution.

In the end, the "best" choice depends on the application!

Filter design techniques:

We will study some of the most popular filter design techniques:

1. Linear-phase FIR filters:
   (a) Window method
   (b) Frequency-sampled method
   (c) Weighted least-squares
   (d) Equiripple (Parks-McClellan)
2. IIR filters:
   (a) Bilinear transform
   (b) Padé's method, Prony's method, and Shank's method

Generalized linear phase:

- Recall that *linear phase* meant

$$H(e^{j\omega}) = A(\omega)e^{-j\omega\ell} \text{ for } A(\omega) \in \mathbb{R} \text{ and } \ell \in \mathbb{Z}.$$

- We can relax this to *generalized linear phase*:

$$\boxed{H(e^{j\omega}) = A(\omega)e^{-j(\omega\ell+\phi_0)} \text{ for } 2\ell \in \mathbb{Z} \text{ and } \phi_0 \in \{0, \tfrac{\pi}{2}\}}$$

  while still retaining the desirable phase behavior.

- Interpretation: The phase derivative or *group delay*

$$g(\omega_0) \triangleq -\frac{\mathrm{d}\phi(\omega)}{\mathrm{d}\omega}\Big|_{\omega=\omega_0}$$

  is the effective delay (in samples) caused by $H(e^{j\omega})$ at frequency ω_0. For GLP, $\phi(\omega) = -\omega\ell - \phi_0$, and so

$$\text{GLP } H(e^{j\omega}) \text{ has } g(\omega_0) = \ell \text{ for all } \omega_0.$$

  Thus, GLP filters delay all frequency components equally!

- As we will now see, the GLP property implies a form of symmetry in the FIR impulse response $h[n]\ldots$

Generalized linear phase (cont.):

- Notice that, for any length-L (order $L-1$) FIR filter,

$$H(e^{j\omega}) = \sum_{n=0}^{L-1} h[n]e^{-j\omega n}$$
$$= e^{-j\omega\frac{L-1}{2}} \sum_{n=0}^{L-1} h[n]e^{-j\omega(n-\frac{L-1}{2})}$$
$$= e^{-j\omega\frac{L-1}{2}} \left[h[0]e^{j\omega\frac{L-1}{2}} + h[1]e^{j\omega(\frac{L-1}{2}-1)} \cdots h[L-1]e^{-j\omega\frac{L-1}{2}} \right].$$

  Since $e^{j\theta} = \cos(\theta) + j\sin(\theta)$ and $e^{-j\theta} = \cos(\theta) - j\sin(\theta)$,

$$\begin{aligned} H(e^{j\omega}) \;=\; & e^{-j\omega\frac{L-1}{2}} \Big[(h[0] + h[L-1])\cos(\omega\tfrac{L-1}{2}) \\ & + j(h[0] - h[L-1])\sin(\omega\tfrac{L-1}{2}) \\ & + (h[1] + h[L-2])\cos(\omega\tfrac{L-3}{2}) \\ & + j(h[1] - h[L-2])\sin(\omega\tfrac{L-3}{2}) + \ldots \Big]. \end{aligned}$$

- To keep $A(\omega)$ in $H(e^{j\omega}) = A(\omega)e^{-j(\omega d+\phi_0)}$ real, we need:

  $\underline{\phi_0 = 0}$:

$$\forall n : \left\{ \begin{array}{l} h[n] + h[L-1-n] \;\in \mathbb{R} \\ h[n] - h[L-1-n] \;\in \mathbb{I} \end{array} \right\} \Leftrightarrow h[n] = h^*[L-1-n]$$

  conjugate symmetry around $\frac{L-1}{2}$.

  $\underline{\phi_0 = \frac{\pi}{2}}$:

$$\forall n : \left\{ \begin{array}{l} h[n] + h[L-1-n] \;\in \mathbb{I} \\ h[n] - h[L-1-n] \;\in \mathbb{R} \end{array} \right\} \Leftrightarrow h[n] = -h^*[L-1-n]$$

  conjugate anti-symmetry around $\frac{L-1}{2}$.

- Often we're interested in real-valued $h[n]$. In this case, GLP implies *symmetry* or *anti-symmetry* around the point $\frac{L-1}{2}$.

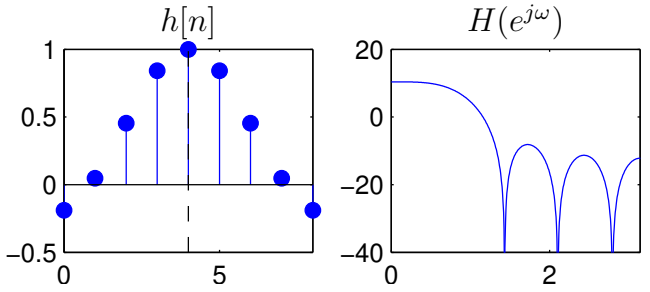
Generalized linear phase (cont.):

There are some implications to symmetry and anti-symmetry.

Consider that $\begin{cases} \text{DC gain: } H(e^{j0}) = \sum_{n=0}^{L-1} h[n], \\ \text{HF gain: } H(e^{j\pi}) = \sum_{n=0}^{L-1} h[n](-1)^n. \end{cases}$

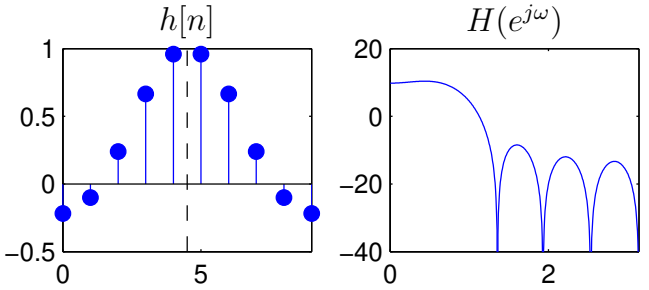
As a consequence, we have four "types" of *real*-GLP filter:

I. odd-length symmetric ($\phi_0 = 0$)



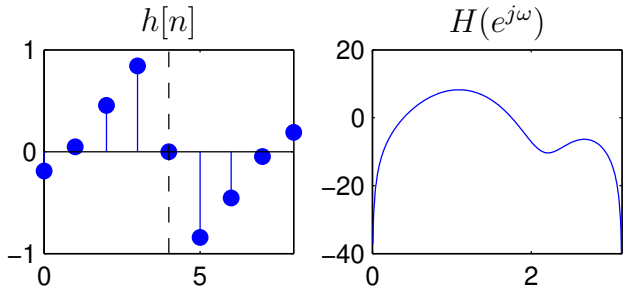
no constraints

II. even-length symmetric ($\phi_0 = 0$)



HF gain = 0!

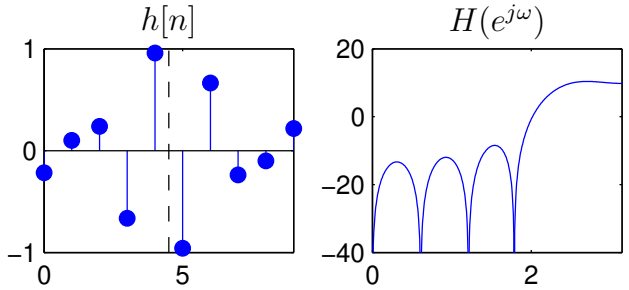
III. odd-length anti-symmetric ($\phi_0 = \frac{\pi}{2}$)



DC gain = 0!

HF gain = 0!

IV. even-length anti-symmetric ($\phi_0 = \frac{\pi}{2}$)



DC gain = 0!

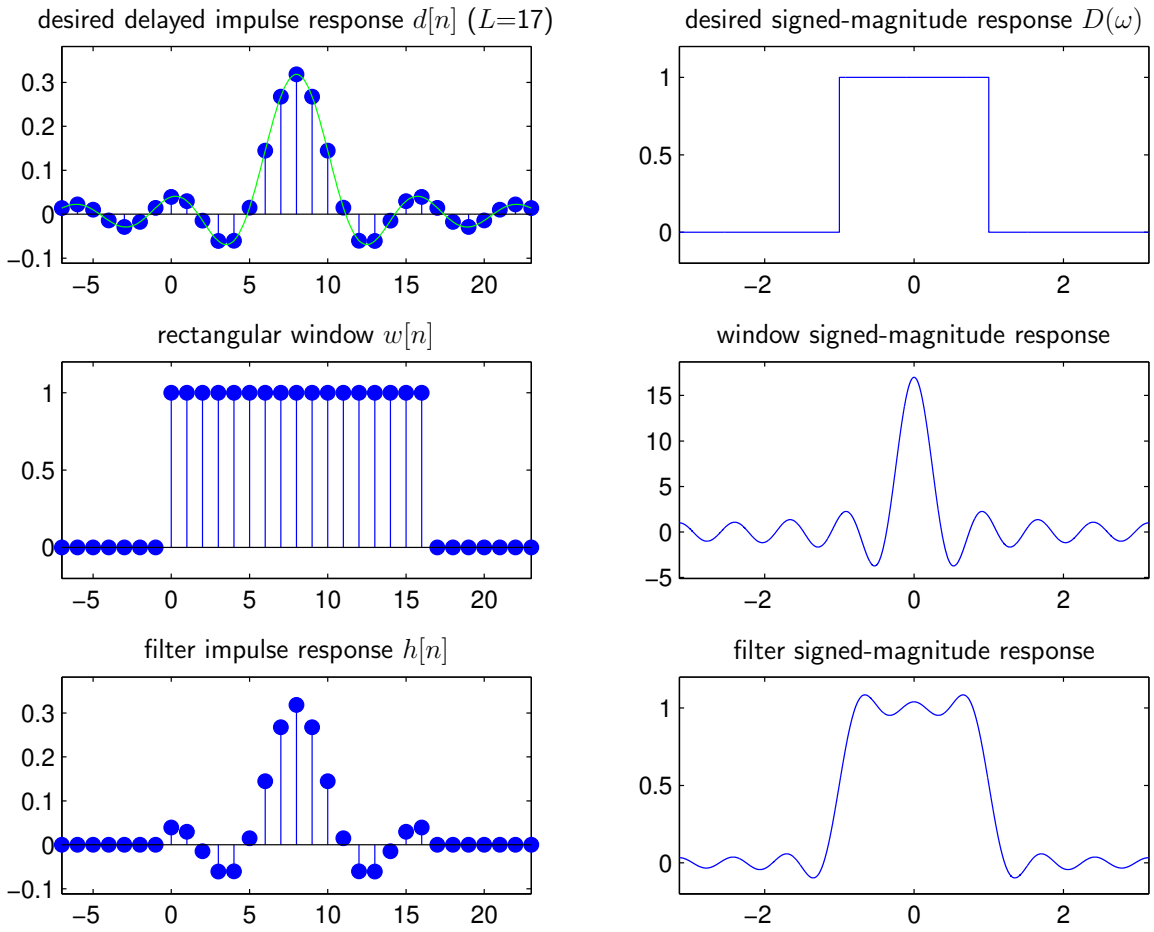
Note: group delay $= \frac{L-1}{2}$ whether length L is even or odd.

Window-based FIR design:

To design a (causal) length-L GLP filter $h[n]$,

1. Specify desired signed-magnitude response $D(\omega) \in \mathbb{R}$,
2. Derive desired $\frac{L-1}{2}$-delayed impulse response:
$$d[n] = \mathcal{F}_{\text{DTFT}}^{-1}\left\{D(\omega)e^{-j(\omega\frac{L-1}{2}+\phi_0)}\right\},\ n \in \mathbb{Z}$$
3. Apply causal length-L window $w[n]$ to $d[n]$:
$$h[n] = w[n]d[n].$$

Recall: time-domain windowing $\Leftrightarrow$ freq-domain convolution.



Window-based FIR design (cont.):

Through choice of window, we can trade off between:

1. transition-band width $\Delta_\omega = |\omega_p - \omega_s|$ (in rad/sample)
2. passband/stopband deviation (Note: $\delta_p = \delta_s$ here!)

Examples for several windows:

window	$\Delta_\omega/2\pi$	$\delta_p = \delta_s$	A_p dB	A_s dB
rectangular	$0.9/L$	0.09	0.82	21
Hann	$3.1/L$	0.0063	0.055	44
Hamming	$3.3/L$	0.0022	0.019	53
Blackman	$5.5/L$	0.0002	0.0017	74

The Kaiser window gives a direct way to attain a given A_s through the choice of its window parameter β:

$$\beta \;=\; \begin{cases} 0 & A_s \le 21 \\ 0.5842(A_s - 21)^{0.4} + 0.07886(A_s - 21) & 21 < A_s \le 50 \\ 0.1102(A_s - 8.7) & 50 < A_s \end{cases}$$

and to attain a given (A_s, Δ_ω) through choice of length L:

$$L \;\approx\; \frac{A_s - 7.95}{2.285\Delta_\omega} + 1.$$

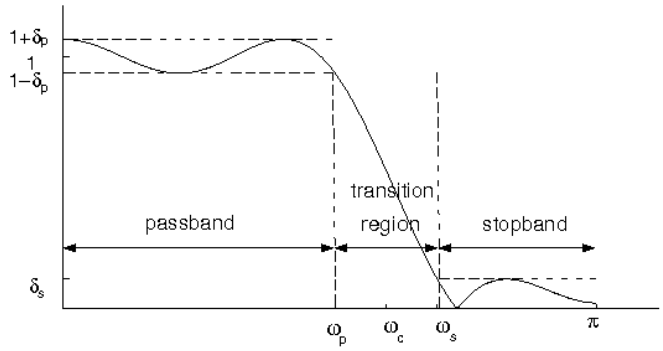
Selection of (β, L) is automated by Matlab's `kaiserord`.

To optimize window designs, a bit of tweaking may be needed!

Example of window-based FIR design:

- Say that we desire a linear-phase LPF with

$$\omega_p = 0.2\pi,\ \omega_s = 0.3\pi,\ \delta_p \le 0.02,\ \delta_s \le 0.01.$$



- For window-based filters, $\delta_p = \delta_s$. We choose $\delta_p = \delta_s = 0.01$, or $A_s = -20\log_{10}(0.01) = 40$ dB, to meet both δ constraints.
- Based on the δ values in the table, the rectangular window won't work, but any of the others will. Of those, the Hann window has sharpest transition. Kaiser would also work...
- For Hann, $\frac{\Delta_w}{2\pi} = \frac{3.1}{L}$. Since $\Delta_\omega = 0.1\pi$, we need $L \approx 62$.
  For Matlab, we use the "cutoff" freq $\omega_c = \frac{\omega_p+\omega_s}{2}$ and type

  `h = fir1(`$L-1$`,`ω_c/π`,hann(`L`),'noscale');`

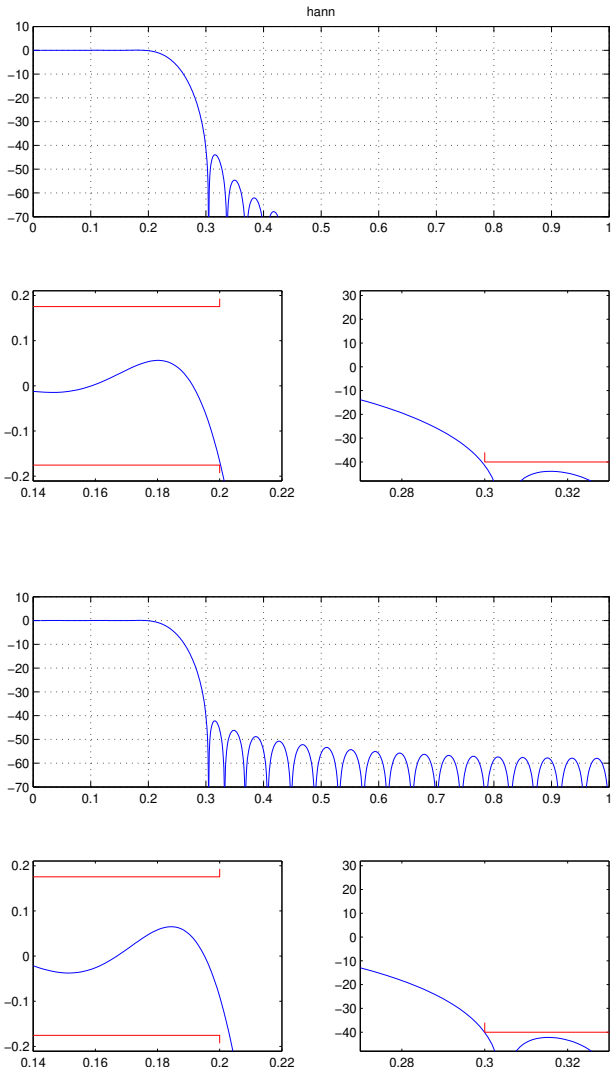
  - $L-1$ specifies the filter *order*
  - ω_c/π is the "Matlab-normalized" cutoff frequency (a Matlab frequency of "1" = Nyquist = π rad/samp)
  - 'noscale' disables scaling of passband midpoint to 0 dB.
- For Kaiser, we can use Matlab's automated design procedure:

```
[N,fc,beta,type]=kaiserord([0.2,0.3],[1,0],[0.02,0.01]);
h=fir1(N,fc,type,kaiser(N+1,beta),'noscale');
```

Example of window-based FIR design (cont.):

The following code designs a filter and plots its DTFT magnitude, zooming into the passband and stopband edges.

```
fp = 0.2;  % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

Ap = -20*log10(1-delp);
As = -20*log10(dels);
delm = min(delp,dels);
Asm = -20*log10(delm);
fc = (fp+fs)/2;

L=60;
h=fir1(L-1,0.992*fc,hann(L));

%[N,fc,beta,type]=kaiserord([0.2,0.3],[1,0],[0.02,0.01]);
%h=fir1(N,fc,type,kaiser(N+1,beta),'noscale');

[H,omega]=freqz(h,1,1024);
f = omega/pi;
subplot(2,1,1);
  plot(f,20*log10(abs(H)));
  grid on;
  axis([0,1,-70,10])
subplot(2,2,3);
  plot(f,20*log10(abs(H)),...
       [0,fp,fp],-Ap*[1,1,1.1],'r',...
       [0,fp,fp],Ap*[1,1,1.1],'r');
  axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
  plot(f,20*log10(abs(H)),...
       [fs,fs,1],-As*[0.9,1,1],'r');
  axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```



For Hann (but not Kaiser), ω_c was tweaked & L minimized!

Frequency-sampled FIR design:

- Given desired signed-magnitude response $D(\omega)$, it's possible to design length-L GLP $h[n]$ that *exactly* equals the desired frequency response at L frequencies $\{\omega_k\}_{k=0}^{L-1}$.
- For a length-L GLP filter, the desired freq response is

$$D(e^{j\omega}) = D(\omega)e^{-j(\omega\frac{L-1}{2}+\phi_0)},$$

and so matching it at frequencies $\{\omega_k\}_{k=0}^{L-1}$ means

$$D(e^{j\omega_k}) = H(e^{j\omega_k}) = \sum_{n=0}^{L-1} h[n]e^{-j\omega_k n} \text{ for } k = 0 \ldots L-1.$$

This gives a system of L linear equations in L unknowns:

$$\underbrace{\begin{bmatrix} D(e^{j\omega_0}) \\ \vdots \\ D(e^{j\omega_{L-1}}) \end{bmatrix}}_{\boldsymbol{d}} = \underbrace{\begin{bmatrix} e^{-j\omega_0 0} & \cdots & e^{-j\omega_0(L-1)} \\ \vdots & \ddots & \vdots \\ e^{-j\omega_{L-1}0} & \cdots & e^{-j\omega_{L-1}(L-1)} \end{bmatrix}}_{\boldsymbol{W}} \underbrace{\begin{bmatrix} h[0] \\ \vdots \\ h[L-1] \end{bmatrix}}_{\boldsymbol{h}}$$

which can be solved via:

$$\boldsymbol{h} = \boldsymbol{W}^{-1}\boldsymbol{d}.$$

- If the frequencies are chosen such that $\omega_k = \frac{2\pi}{L}k$, we get

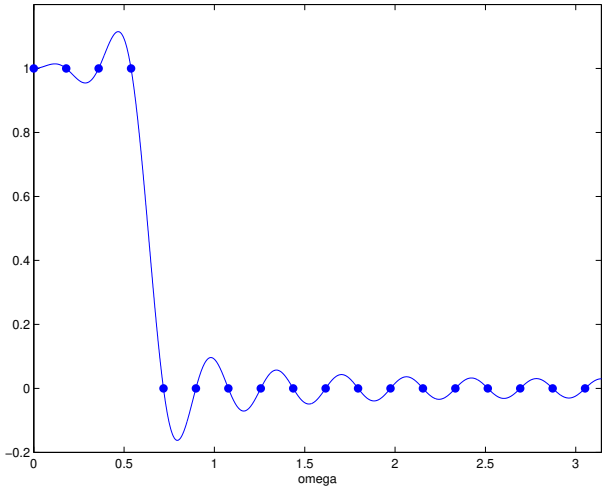
$$\boldsymbol{h} = \boldsymbol{W}_{L\text{-DFT}}^{-1}\boldsymbol{d}$$

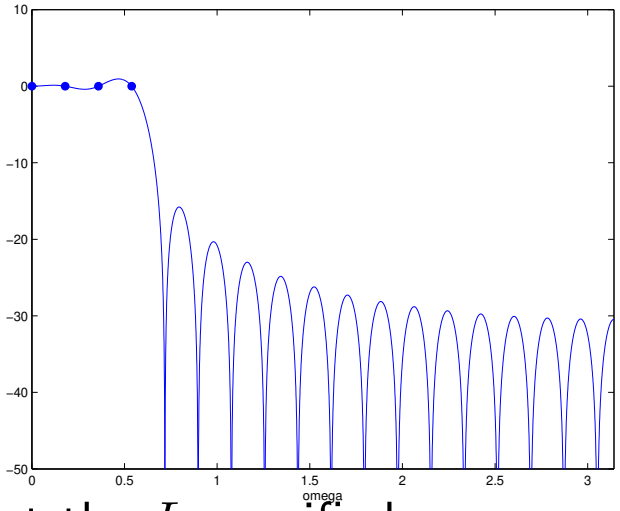
which can be calculated efficiently using an L-IFFT.

Example of frequency-sampled FIR design:

The following code designs a length-35 GLP LPF with cutoff frequency $\omega_c = \frac{2\pi}{10}$, and plots both linear signed-magnitude response and dB-magnitude response.

```
L = 35;
phi0 = 0;
omega_c = 0.1*2*pi;
omega = 2*pi*[0:L-1]/L;
pass1 = find(omega<omega_c);
pass2 = find(omega>2*pi-omega_c);
D = zeros(1,L);
D(pass1) = exp(-j*((L-1)/2*omega(pass1)+phi0));
D(pass2) = exp(-j*((L-1)/2*omega(pass2)+phi0));
h = real(ifft(D));

M = 8192;
H = real(fft(h,M).*exp(j*((L-1)/2*[0:M-1]/M*2*pi+phi0)));
figure(1); clf;
plot([0:M-1]/M*2*pi,H,[0:L-1]/L*2*pi,abs(D),'.b');
axis([0,pi,-0.2,1.2])
xlabel('omega')
figure(2); clf;
plot([0:M-1]/M*2*pi,20*log10(abs(H)),...
     [0:L-1]/L*2*pi,20*log10(abs(D)),'.b');
axis([0,pi,-50,10])
xlabel('omega')
```





While the response is indeed perfect at the L specified frequencies, it is not very good inbetween!

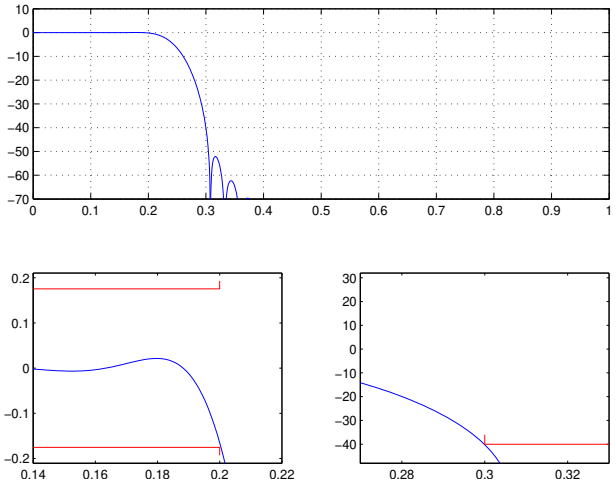
Performance can be improved by windowing the resulting impulse response, as automated by Matlab's `fir2` ...

Example of `fir2`-based FIR design:

`fir2` performs frequency-sampled FIR design with automatic impulse-response windowing (default is Hamming).

Let's try `fir2` on the same LPF design considered earlier:

```
fp = 0.2;  % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

fc = (fp+fs)/2;
L=68;
h=fir2(L-1,[0,0.991*fc,0.991*fc,1],[1,1,0,0],hann(L));



Ap = -20*log10(1-delp);
As = -20*log10(dels);
[H,omega]=freqz(h,1,1024);
f = omega/pi;
subplot(2,1,1);
  plot(f,20*log10(abs(H)));
  grid on;
  axis([0,1,-70,10])
subplot(2,2,3);
  plot(f,20*log10(abs(H)),...
       [0,fp,fp],-Ap*[1,1,1.1],'r',...
       [0,fp,fp],Ap*[1,1,1.1],'r');
  axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
  plot(f,20*log10(abs(H)),...
       [fs,fs,1],-As*[0.9,1,1],'r');
  axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```



Even after tweaking, `fir2` requires a longer filter! :(

The advantage to `fir2` is that it works with *any* $D(e^{j\omega})$; no complicated steps like $\mathcal{F}^{-1}_{\text{DTFT}}\{D(e^{j\omega})\}$ are needed!

Least-squares FIR design:

- Given desired signed-magnitude response $D(\omega)$, we will now design the length-L GLP filter $h[n]$ that minimizes the squared response error

$$\mathcal{E}_2 = \frac{1}{2\pi}\int_{-\pi}^{\pi} \left|D(e^{j\omega}) - H(e^{j\omega})\right|^2 \mathrm{d}\omega.$$

- For a length-L GLP filter, the desired freq response is

$$D(e^{j\omega}) = D(\omega)e^{-j(\omega\frac{L-1}{2}+\phi_0)},$$

from which we can obtain the desired impulse response

$$d[n] = \mathcal{F}^{-1}_{\text{DTFT}}\{D(e^{j\omega})\}.$$

- Parseval's theorem implies that

$$\begin{aligned}\mathcal{E}_2 &= \sum_{n=-\infty}^{\infty} \left|d[n] - h[n]\right|^2 \\ &= \sum_{n=-\infty}^{-1} \left|d[n]\right|^2 + \sum_{n=0}^{L-1} \left|d[n] - h[n]\right|^2 + \sum_{n=L}^{\infty} \left|d[n]\right|^2\end{aligned}$$

which is minimized by

$$h[n] = d[n] \text{ for } n = 0, \ldots, L-1.$$

$\Rightarrow$ *rectangular-window method is least-squares optimal!*

- But rectangular-window designs have very large ripples! Why are they least-squares optimal? The LS criterion doesn't include don't-care regions, and so the filter worries most about transition-band error.

Weighted least-squares FIR design:

- The weighted-LS criterion allows more control over ripple performance:

$$\mathcal{E}_W = \frac{1}{2\pi}\int_{-\pi}^{\pi} W(\omega)\left|D(e^{j\omega}) - H(e^{j\omega})\right|^2 \mathrm{d}\omega.$$

- For length-L GLP, $D(e^{j\omega}) = D(\omega)e^{-j(\omega\frac{L-1}{2}+\phi_0)}$, so

$$\begin{aligned}\mathcal{E}_W &= \frac{1}{2\pi}\int_{-\pi}^{\pi} W(\omega)\left|D(\omega)e^{-j(\omega\frac{L-1}{2}+\phi_0)} - H(e^{j\omega})\right|^2 \mathrm{d}\omega \\ &= \frac{1}{2\pi}\int_{-\pi}^{\pi} W(\omega)\left|D(\omega) - H(e^{j\omega})e^{j(\omega\frac{L-1}{2}+\phi_0)}\right|^2 \mathrm{d}\omega.\end{aligned}$$

- For <u>Type-I</u> GLP filters (odd L, $\phi_0 = 0$, symmetric $h[n]$):

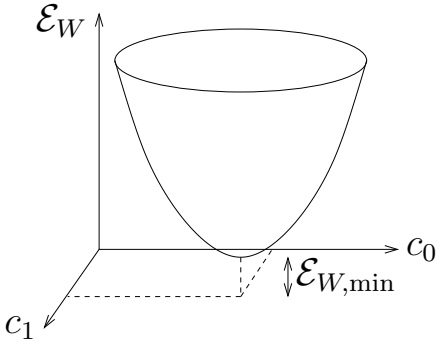
$$\begin{aligned}H(e^{j\omega})e^{j(\omega\frac{L-1}{2}+\phi_0)} &= \sum_{n=0}^{L-1} h[n]e^{-j\omega(n-\frac{L-1}{2})} \\ &= (h[0]+h[L-1])\cos(\omega\tfrac{L-1}{2}) + j(h[0]-h[L-1])\sin(\omega\tfrac{L-1}{2}) \\ &\quad + (h[1]+h[L-2])\cos(\omega\tfrac{L-3}{2}) + j(h[1]-h[L-2])\sin(\omega\tfrac{L-3}{2}) \\ &\quad + \cdots + h[(L-1)/2] \\ &= h[(L-1)/2] + \sum_{n=0}^{\frac{L-1}{2}-1} 2h[n]\cos(\omega(\tfrac{L-1}{2}-n)) \quad \text{for real-valued } h[n] \\ &= \sum_{n=0}^{\frac{L-1}{2}} c_n \cos(\omega(\tfrac{L-1}{2}-n)) \quad \text{with} \quad c_n = \begin{cases} 2h[n] & \text{for } n = 0\ldots\frac{L-1}{2}-1 \\ h[n] & \text{for } n = \frac{L-1}{2}\end{cases}\end{aligned}$$

Weighted least-squares FIR design (cont.):

For real length-L Type-I GLP filters, the WLS criterion is:

$$\mathcal{E}_W \;=\; \frac{1}{2\pi}\int_{-\pi}^{\pi} W(\omega)\left[D(\omega) - \sum_{n=0}^{(L-1)/2} c_n \cos\bigl(\omega(\tfrac{L-1}{2}-n)\bigr)\right]^2 \mathrm{d}\omega$$

which is a quadratic function of the coefficients $\{c_n\}_{n=0}^{(L-1)/2}$.



At the optimal solution, the derivatives $\{\frac{\partial \mathcal{E}_W}{\partial c_l}\}_{l=0}^{(L-1)/2}$ must equal zero.

These derivatives are:

$$\frac{\partial \mathcal{E}_W}{\partial c_l} \;=\; \frac{1}{2\pi}\int_{-\pi}^{\pi} W(\omega)\frac{\partial}{\partial c_l}\left[D(\omega) - \sum_{n=0}^{(L-1)/2} c_n \cos\bigl(\omega(\tfrac{L-1}{2}-n)\bigr)\right]^2 \mathrm{d}\omega$$

$$= \;\frac{-2}{2\pi}\int_{-\pi}^{\pi} W(\omega)\cos\bigl(\omega(\tfrac{L-1}{2}-l)\bigr)\left[D(\omega) - \sum_{n=0}^{(L-1)/2} c_n \cos\bigl(\omega(\tfrac{L-1}{2}-n)\bigr)\right]\mathrm{d}\omega$$

$$= \;-\frac{1}{\pi}\int_{-\pi}^{\pi} W(\omega)\cos\bigl(\omega(\tfrac{L-1}{2}-l)\bigr)D(\omega)\,\mathrm{d}\omega$$
$$+\sum_{n=0}^{(L-1)/2} c_n \;\frac{1}{\pi}\int_{-\pi}^{\pi} W(\omega)\cos\bigl(\omega(\tfrac{L-1}{2}-l)\bigr)\cos\bigl(\omega(\tfrac{L-1}{2}-n)\bigr)\,\mathrm{d}\omega.$$

Zeroing gives $\frac{L-1}{2}+1$ linear equations in $\frac{L-1}{2}+1$ unknowns:

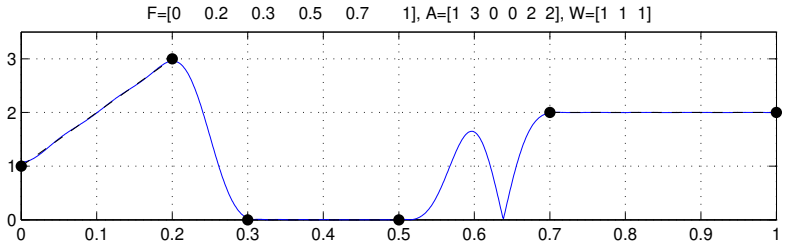
$$0 = -p_l + \sum_{n=0}^{(L-1)/2} c_n Q_{l,n} \text{ for } l = 0\ldots\tfrac{L-1}{2} \quad\Leftrightarrow\quad \boldsymbol{p} = \boldsymbol{Q}\boldsymbol{c}$$

which can be solved via $\boldsymbol{c} = \boldsymbol{Q}^{-1}\boldsymbol{p}$.

Example of weighted-LS FIR design:

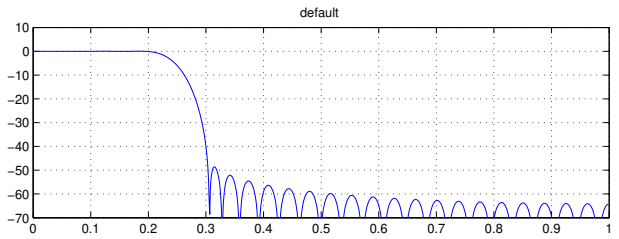
Matlab's `firls(`$L-1$`,F,A,W)` performs WLS optimization:

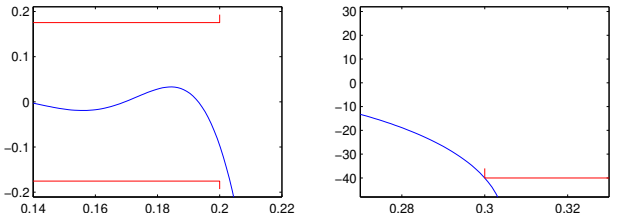
- `F`: frequencies (pairs)
- `A`: amplitudes (pairs)
- `W`: weights (singles)

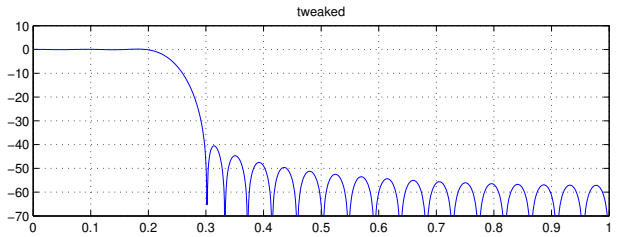


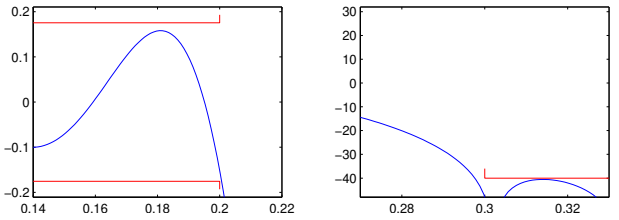
The following code designs a WLS LPF and then tweaks it:

```
fp = 0.2;  % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

%L=53;
%F=[0,fp,fs,1]; A=[1,1,0,0]; W=[1/delp,1/dels];
%h=firls(L-1,F,A,W);

L=44;
F=[0,1.035*fp,0.975*fs,1]; A=[1,1,0,0]; W=[1/delp,2/dels]
h=firls(L-1,F,A,W);

Ap = -20*log10(1-delp);
As = -20*log10(dels);

[H,omega]=freqz(h,1,1024);
f = omega/pi;
subplot(2,1,1);
  plot(f,20*log10(abs(H)));
  grid on;
  axis([0,1,-70,10])
subplot(2,2,3);
  plot(f,20*log10(abs(H)),...
       [0,fp,fp],-Ap*[1,1,1.1],'r',...
       [0,fp,fp],Ap*[1,1,1.1],'r');
  axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
  plot(f,20*log10(abs(H)),...
       [fs,fs,1],-As*[0.9,1,1],'r');
  axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```



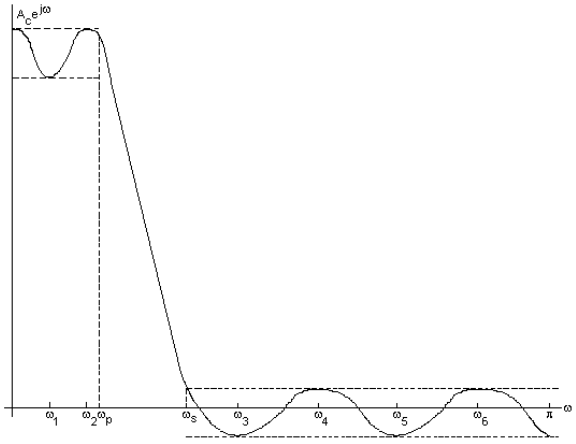






Equiripple (Parks-McClellan) FIR design:

- If we want to minimize the peak deviation from the desired $D(\omega)$, then *equiripple* filters are optimal.
- Intuitively speaking, when we try to suppress one ripple, the others get larger. Imagine repeatedly re-designing a WLS filter with increased weight around the worst ripple. We would eventually get a filter with equal-sized ripples:

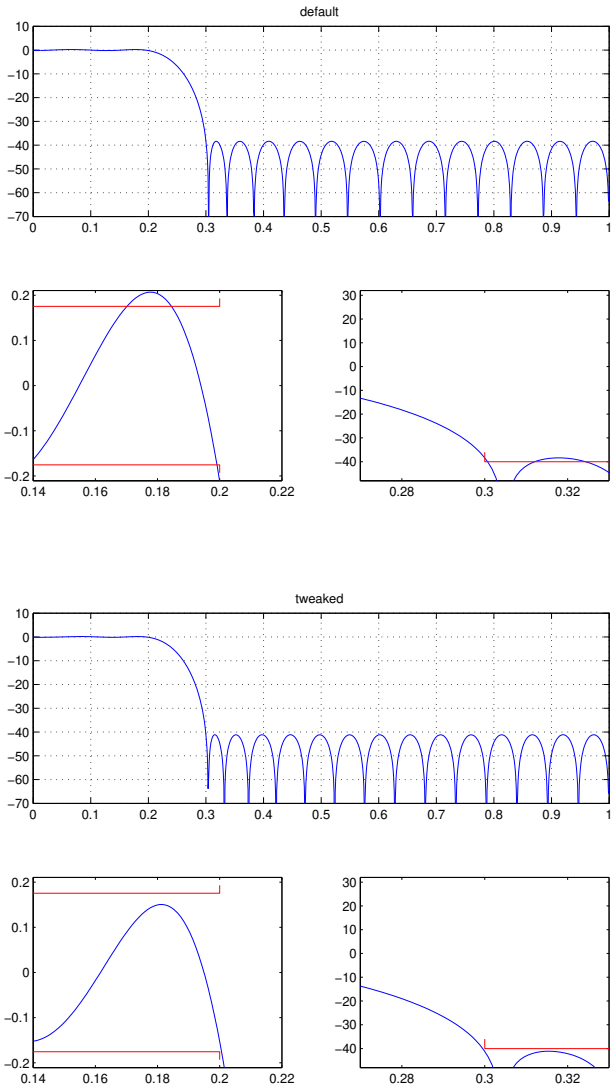


- The Parks-McClellan algorithm accomplishes this task, but in a much faster way, using the Chebyshev's alternation theorem and the Remez exchange algorithm.
- `firpm(`$L-1$`,F,A,W)` performs Matlab equiripple design, with the same interface as `firls(...)`.
- The command `firpmord` aids in the selection of the `firpm` input parameters.

Example of equiripple FIR design:

The following code designs an equiripple LPF using `firpm` with inputs generated using `firpmord`.

```
fp = 0.2;  % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

[N,F,A,W]=firpmord([fp,fs],[1,0],[delp,dels])
%h=firpm(N,F,A,W);
h=firpm(N+2,F,A,W);

Ap = -20*log10(1-delp);
As = -20*log10(dels);

[H,omega]=freqz(h,1,1024);
f = omega/pi;
subplot(2,1,1);
  plot(f,20*log10(abs(H)));
  grid on;
  axis([0,1,-70,10])
subplot(2,2,3);
  plot(f,20*log10(abs(H)),...
       [0,fp,fp],-Ap*[1,1,1.1],'r',...
       [0,fp,fp],Ap*[1,1,1.1],'r');
  axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
  plot(f,20*log10(abs(H)),...
       [fs,fs,1],-As*[0.9,1,1],'r');
  axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```



The order $N = 35$ guessed by `firpmord` was a bit too small, and so it was increased by 2 to meet the specs.

Summary of FIR filter design:

For GLP FIR filter design, we examined:

design method	matlab command(s)	example length L
window-based	`fir1` `kaiserord`	60 (Hann) 46 (Kaiser)
freq-sampled	`fir2`	68 (Hann)
weighted least-squares	`firls`	44
equiripple	`firpm` `firpmord`	38

where the "example length" was the minimum required to design a particular LPF under peak ripple constraints.

IIR filters:

- Recall that we have two ways to specify the transfer function of an IIR (i.e., recursive) filter:

  1. A ratio of two polynomials in z^{-1}:

$$H(z) = \frac{\sum_{k=0}^{N} b_k z^{-k}}{1 + \sum_{m=1}^{M} a_m z^{-m}}$$

  2. A ratio of two factored polynomials in z^{-1}:

$$H(z) = \frac{b_0 \prod_{k=1}^{N} (1 - z_k z^{-1})}{\prod_{m=1}^{M} (1 - p_m z^{-1})} \text{ with } \begin{cases} \text{zeros } \{z_k\}_{k=1}^{N} \\ \text{poles } \{p_m\}_{m=1}^{M} \end{cases}$$

  In either case, the "order" $= \max(M, N)$.

- $H(z)$ is *(BIBO) stable* iff all poles are inside the unit circle: $|p_m| < 1$ for all m.

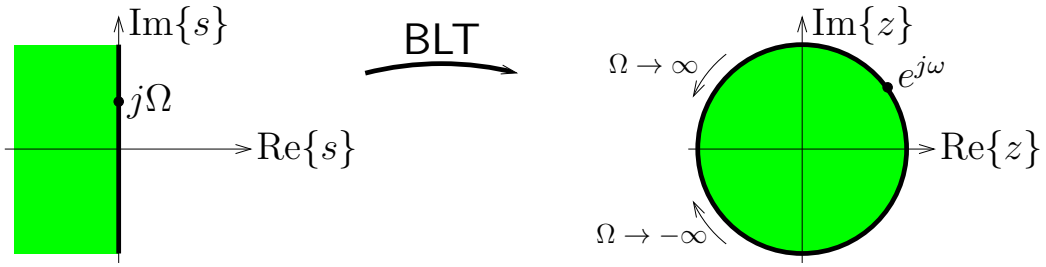
- Also recall that the poles and zeros come in complex-conjugate pairs (and/or real-valued singletons) iff $\{b_k\}$ and $\{a_m\}$ are real-valued.

- Next we'll talk about ways of designing the filter coefficients...

Bilinear transform:

- One method of designing IIR digital filters is to start with an analog filter and "map" it to the digital domain.
- The *bilinear transform* accomplishes this mapping:

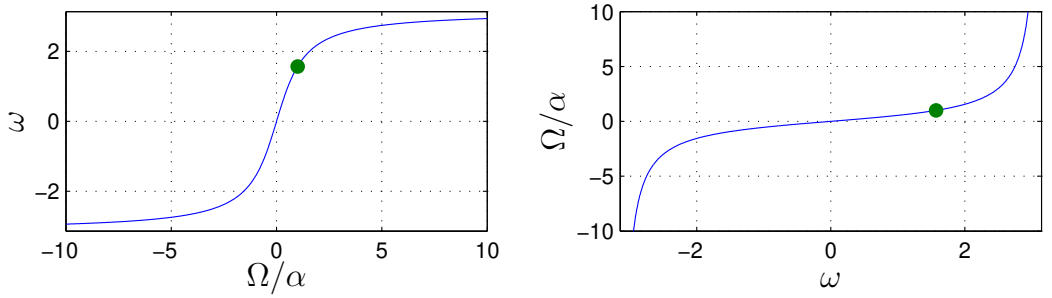
$$H(z) = H_c(s)\Big|_{s=\alpha\frac{z-1}{z+1}=\alpha\frac{1-z^{-1}}{1+z^{-1}}}$$



  - The s-plane stability region/boundary is mapped to the z-plane stability region/boundary.
  - The point $s = j\Omega$ is mapped to $z = e^{j\omega}$, where

$$\omega = 2\tan^{-1}\frac{\Omega}{\alpha} \quad \text{(e.g., } \Omega = \alpha \leftrightarrow \omega = \tfrac{\pi}{2}\text{)}$$

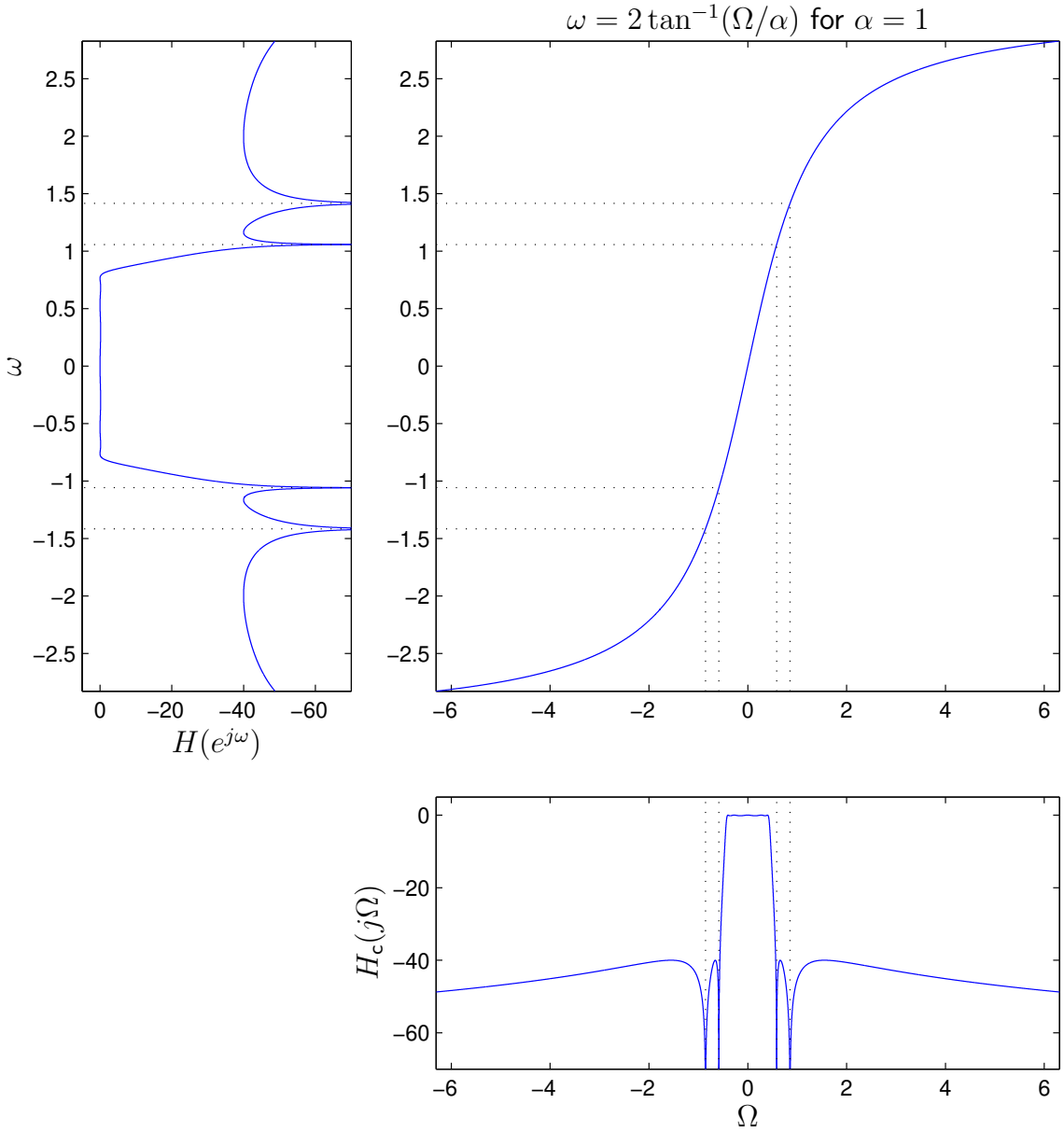
$$\Omega = \alpha\tan\frac{\omega}{2} \quad \text{(e.g., } \omega = \pi \leftrightarrow \Omega = \infty\text{)}$$



As we will see, the choice of α is arbitrary.

Bilinear transform (cont.):

Mapping a LPF from Ω-domain to ω-domain:



- Say ω_c is desired cutoff freq of $H(e^{j\omega})$. Then $H_c(j\Omega)$ is designed with cutoff frequency $\Omega_c = \alpha \tan \frac{\omega_c}{2}$. (Since we can accomodate any Ω_c, the choice of α doesn't matter!)
- Ripple in $H(e^{j\omega})$ will be identical to ripple in $H_c(j\Omega)$.

Analog prototypes:

- The most common analog prototype filters are:

filter type	passband	stopband	matlab	BLT?
Butterworth	smooth	smooth	`butter`	Y
Chebyshev	equiripple	smooth	`cheby1`	Y
	smooth	equiripple	`cheby2`	Y
Cauer (elliptic)	equiripple	equiripple	`ellip`	Y
Bessel	smooth	smooth	`besself`	N

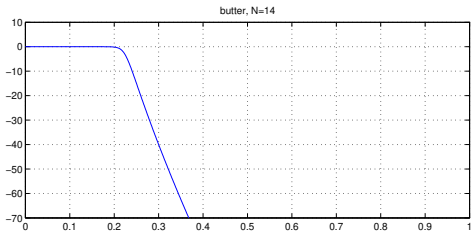
- The above Matlab commands usually include the bilinear transform step, so you just specify the order N, desired cutoff ω_c, and filter type (LPF, HPF, etc.), and—in some cases—the ripple specs. (Note: For BPFs, N is doubled!)
- The corresponding commands `buttord`, `cheb1ord`, `cheb2ord`, and `ellipord` guess the required filter order.
- Note: Bessel analog filters have approximately linear phase through their passband. (Good!) However, they are LPF-only, have a wide transition band, and you have to perform manual BLT from the `besself` output, i.e.,

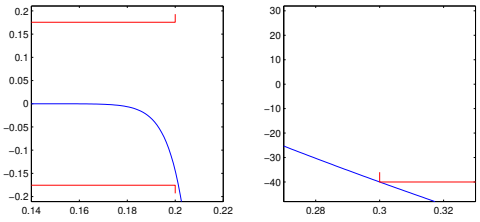
$$H(z) = \left. \frac{B_0 s^N + B_1 s^{N-1} + \cdots + B_N}{s^N + A_1 s^{N-1} + \cdots + A_N} \right|_{s=\alpha \frac{z-1}{z+1}}$$

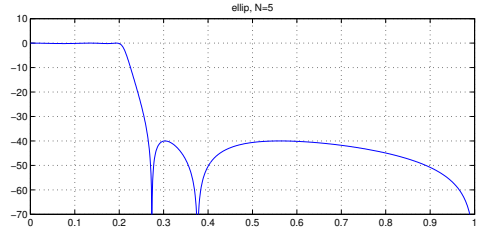
for the $\{B_n\}$ and $\{A_n\}$ returned by `besself`.

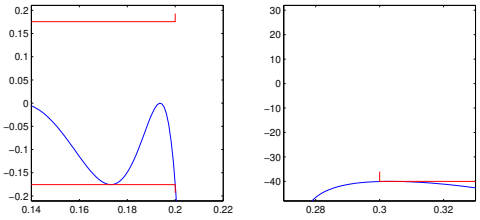
Analog-to-IIR examples:

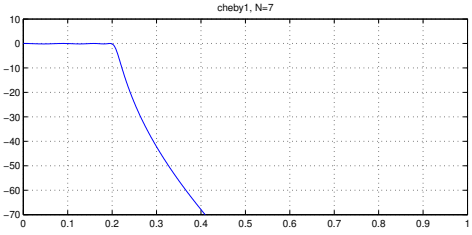
```
fp = 0.2;  % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

Ap = -20*log10(1-delp);
As = -20*log10(dels);

%[N,fc] = buttord(fp,fs,Ap,As); [b,a] = butter(N,fc);
%[N,fc] = cheb1ord(fp,fs,Ap,As); [b,a] = cheby1(N,Ap,fc);
%[N,fc] = cheb2ord(fp,fs,Ap,As); [b,a] = cheby2(N,As,fc);
[N,fc] = ellipord(fp,fs,Ap,As); [b,a] = ellip(N,Ap,As,fc)

[H,omega]=freqz(b,a,1024);
f = omega/pi;
subplot(2,1,1);
  plot(f,20*log10(abs(H)));
  grid on;
  axis([0,1,-70,10])
subplot(2,2,3);
  plot(f,20*log10(abs(H)),...
       [0,fp,fp],-Ap*[1,1,1.1],'r',...
       [0,fp,fp],Ap*[1,1,1.1],'r');
  axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
  plot(f,20*log10(abs(H)),...
       [fs,fs,1],-As*[0.9,1,1],'r');
  axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```

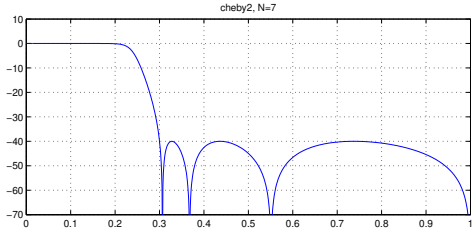


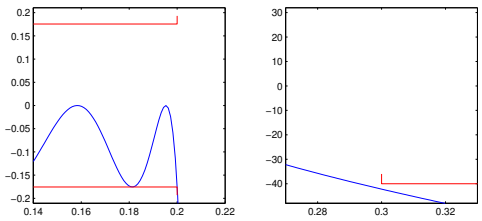


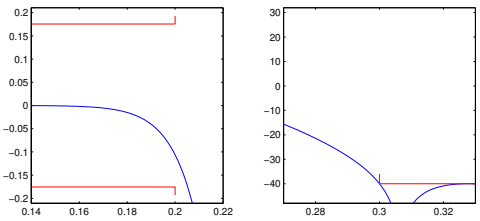






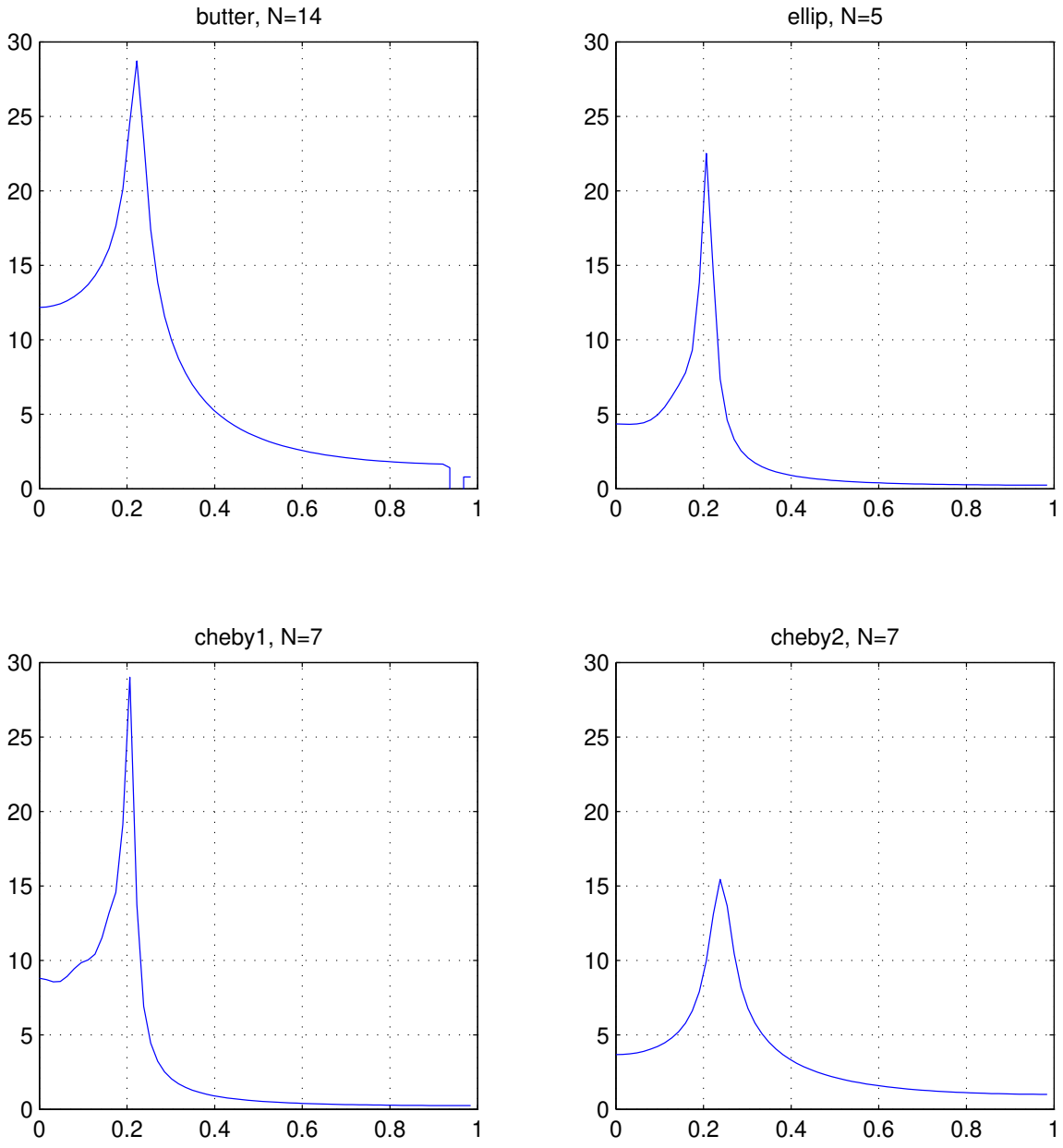






Analog-to-IIR examples (cont.):

Group delay of the filters on the previous page:



The group delay in the passband (which is the region we care about most) is far from flat!

Padé's IIR method:

Consider design of the causal IIR filter:

$$H(z) \;=\; \frac{\sum_{k=0}^{N} b_k z^{-k}}{1+\sum_{m=1}^{M} a_m z^{-m}}$$

Since

$$H(z) \;=\; \sum_{k=0}^{N} b_k z^{-k} - \sum_{m=1}^{M} a_m z^{-m} H(z),$$

we have

$$\begin{aligned} h[n] \;&=\; \textstyle\sum_{k=0}^{N} b_k \delta[n-k] - \sum_{m=1}^{M} a_m h[n-m] \\ &=\; \begin{cases} 0 & n<0 \\ b_n - \sum_{m=1}^{M} a_m h[n-m] & 0 \le n \le N \\ -\sum_{m=1}^{M} a_m h[n-m] & N<n \end{cases} \end{aligned}$$

Main idea: We can *exactly match* $\{h[n]\}$ to a desired causal impulse response $\{d[n]\}$ at indices $n = 0 \ldots N+M$! It is done in two steps:

1. Solve for $\{a_m\}_{m=1}^{M}$ that yield a match at $n = N+1 \ldots N+M$:

$$\underbrace{-\begin{bmatrix} d[N+1] \\ \vdots \\ d[N+M] \end{bmatrix}}_{\boldsymbol{d}_{\text{padé}}} = \underbrace{\begin{bmatrix} d[N] & d[N-1] & \ldots & d[N-M+1] \\ \vdots & \vdots & \ddots & \vdots \\ d[N+M-1] & d[N+M-2] & \ldots & d[N] \end{bmatrix}}_{\boldsymbol{D}_{\text{padé}}} \underbrace{\begin{bmatrix} a_1 \\ \vdots \\ a_M \end{bmatrix}}_{\boldsymbol{a}}$$

$$\Rightarrow \boldsymbol{a}_{\text{padé}} \;=\; \boldsymbol{D}_{\text{padé}}^{-1} \boldsymbol{d}_{\text{padé}}$$

2. Solve for $\{b_n\}_{n=0}^{N}$ that yield a match at $n = 0 \ldots N$:

$$b_n \;=\; d[n] + \sum_{m=1}^{M} a_{\text{padé},m} d[n-m] \text{ for } n = 0 \ldots N.$$

Caveat: We are not even attempting to match $\{d[n]\}$ for $n > N+M$, so the designed response $\{h[n]\}$ might be really bad there!

Prony's method:

To prevent $\{h[n]\}$ and $\{d[n]\}$ from differing wildly when $n > N+M$, we can give up on *exactly* matching $\{d[n]\}_{n=0}^{N+M}$ and instead design $\{a_m\}_{m=1}^{N}$ to *approximately* match $\{d[n]\}_{n=N+1}^{Q}$ for some $Q > N+M$.

If we choose Q large enough, then usually both $d[n] \approx 0$ and $h[n] \approx 0$ for $n > Q$, giving the chance that $d[n] \approx h[n]$ everywhere.

For the approximate match, we can use a least-squares fit in place of the equality used for Padé's method (where $Q = N+M$):

$$\begin{aligned}
\boldsymbol{a}_{\text{prony}} &= \underset{a_1 \ldots a_M}{\operatorname{argmin}} \sum_{n=N+1}^{Q} \left| \big(-d[n]\big) - \sum_{m=1}^{M} d[n-m] a_m \right|^2 \\
&= \underset{\boldsymbol{a}}{\operatorname{argmin}} \left\| \underbrace{\begin{bmatrix} d[N] & \ldots & d[N-M+1] \\ d[N+1] & \ldots & d[N-M+2] \\ \vdots & & \vdots \\ d[Q-1] & \ldots & d[Q-M] \end{bmatrix}}_{\boldsymbol{D}_{\text{prony}}} \begin{bmatrix} a_1 \\ \vdots \\ a_M \end{bmatrix} - \underbrace{\begin{bmatrix} -d[N+1] \\ -d[N+2] \\ \vdots \\ -d[Q] \end{bmatrix}}_{\boldsymbol{d}_{\text{prony}}} \right\|^2 \\
&= (\boldsymbol{D}_{\text{prony}}^H \boldsymbol{D}_{\text{prony}})^{-1} \boldsymbol{D}_{\text{prony}}^H \boldsymbol{d}_{\text{prony}}
\end{aligned}$$

We then solve for $\{b_n\}_{n=0}^{N}$ as we did in Padé's method:

$$b_n \;=\; d[n] + \sum_{m=1}^{M} a_{\text{prony},m} d[n-m] \quad \text{for} \quad n = 0 \ldots N.$$

In summary, Prony's method gives

$$\begin{aligned}
h[n] &= d[n] \quad \text{for} \quad n = 0 \ldots N \\
h[n] &\approx d[n] \quad \text{for} \quad n = N+1 \ldots Q
\end{aligned}$$

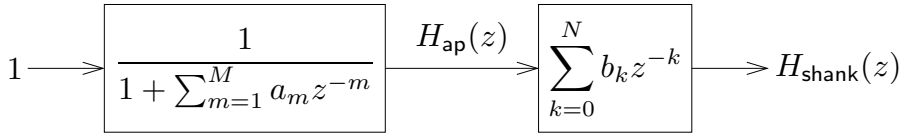
and hopefully $h[n] \approx 0 \approx d[n]$ for $n > Q$.

Shank's IIR method:

Shank's method is an improvement on Prony's method that allows $\{b_k\}_{k=0}^N$, as well as $\{a_m\}_{m=1}^M$, to approximately fit the actual $\{h[n]\}$ to the desired $\{d[n]\}$ over $n = 0 \ldots Q$, for some $Q > N+M$.

Shank's method takes a two-step approach:

1. design the causal all-pole $H_{\text{ap}}(z) = \frac{1}{1+\sum_{m=1}^M a_m z^{-m}}$ that gives the best LS fit to desired impulse response $\{d[n]\}$ over $n = 1 \ldots Q$, then
2. design $\{b_k\}_{k=0}^N$ to best improve $\{h_{\text{ap}}[n]\}$, again using LS.



Step 1)

Similar to Prony (but fit $\{d[n]\}_{n=1}^Q$ rather than $\{d[n]\}_{n=N+1}^Q$):

$$\boldsymbol{a}_{\text{shank}} = \underset{\boldsymbol{a}}{\text{argmin}} \left\| \underbrace{\begin{bmatrix} d[0] & 0 & 0 \\ \vdots & \ddots & 0 \\ d[Q-M] & & d[0] \\ \vdots & \ddots & \vdots \\ d[Q-1] & \ldots & d[Q-M] \end{bmatrix}}_{\boldsymbol{D}_{\text{shank}}} \begin{bmatrix} a_1 \\ \vdots \\ a_M \end{bmatrix} - \underbrace{\begin{bmatrix} -d[1] \\ \vdots \\ -d[Q-M+1] \\ \vdots \\ -d[Q] \end{bmatrix}}_{\boldsymbol{d}_{\text{shank}}} \right\|^2$$

$$= (\boldsymbol{D}_{\text{shank}}^H \boldsymbol{D}_{\text{shank}})^{-1} \boldsymbol{D}_{\text{shank}}^H \boldsymbol{d}_{\text{shank}}$$

This yields (from block diagram)

$$H_{\text{ap}}(z) = 1 - \sum_{m=1}^M a_{\text{shank},m} z^{-m} H_{\text{ap}}(z)$$

$$\Rightarrow h_{\text{ap}}[n] = \begin{cases} 0 & n < 0 \\ 1 & n = 0 \\ -\sum_{m=1}^M a_{\text{shank},m} h_{\text{ap}}[n-m] & n > 0 \end{cases}$$

Shank's IIR method (cont.):

Step 2)

From block diagram, we see that

$$H_{\text{shank}}(z) = \sum_{k=0}^{N} b_k z^{-k} H_{\text{ap}}(z)$$

$$\Rightarrow h_{\text{shank}}[n] = \sum_{k=0}^{N} b_k h_{\text{ap}}[n-k]$$

and so, to yield the best LS fit of $\{h_{\text{shank}}[n]\}$ to $\{d[n]\}$ over $n = 0 \ldots Q$, we solve for

$$\boldsymbol{b}_{\text{shank}} = \underset{b_0 \ldots b_N}{\text{argmin}} \sum_{n=0}^{Q} \left| d[n] - \sum_{k=0}^{N} b_k h_{\text{ap}}[n-k] \right|^2$$

$$= \underset{\boldsymbol{b}}{\text{argmin}} \left\| \underbrace{\begin{bmatrix} h_{\text{ap}}[0] & 0 & 0 \\ \vdots & \ddots & 0 \\ h_{\text{ap}}[Q-N] & & h_{\text{ap}}[0] \\ \vdots & \ddots & \vdots \\ h_{\text{ap}}[Q] & \ldots & h_{\text{ap}}[Q-N] \end{bmatrix}}_{\boldsymbol{H}_{\text{ap}}} \underbrace{\begin{bmatrix} b_0 \\ \vdots \\ b_N \end{bmatrix}}_{\boldsymbol{b}} - \underbrace{\begin{bmatrix} d[0] \\ \vdots \\ d[Q-N] \\ \vdots \\ d[Q] \end{bmatrix}}_{\boldsymbol{d}} \right\|^2$$

$$= (\boldsymbol{H}_{\text{ap}}^H \boldsymbol{H}_{\text{ap}})^{-1} \boldsymbol{H}_{\text{ap}}^H \boldsymbol{d}$$